

COMPLEX ENGINEERING PROBLEM EL-408 (VLSI System Design)

Muhammad Abdul Ahad (EL – 20136)
Sec 'A'

ABSTRACT

This project focuses on the "Design and Implementation of a Verilog-based Round-Robin Arbiter with Fixed-Time Slice Allocation." The need for equitable resource allocation in digital systems is addressed through the implementation of a Round Robin Arbiter in Verilog. The arbiter, handling 4 input requests, employs a ONE HOT Round Robin scheduling scheme. The project explores the Round Robin Algorithm's versatility, emphasizing its applications in CPU scheduling, shared bus arbitration, queue management, and workload balancing. Challenges related to process starvation and considerations for fair resource allocation are discussed. The design choice, structure of the arbiter module, and the flow of data are detailed, emphasizing the practicality of using case statements. The Verilog HDL code is explained, illustrating the Finite State Machine logic. Test bench setup and waveform analysis confirm the arbiter's effectiveness in providing equal access to resources. The conclusion affirms that the design successfully meets the expectations of a Round Robin Arbiter, addressing the stated problem of fair and controlled resource allocation.

VLSI SYSTEM DESIGN (EL-408)

COMPLEX ENGINEERING PROBLEM (CEP)

Project Title: Design and Implementation of a Verilog-based Round-Robin Arbiter with Fixed-Time Slice Allocation

Problem Statement:

Digital systems often require fair resource allocation among competing requests. Implementing a round-robin arbiter using Verilog for fixed-time slice allocation addresses this need by providing an equitable and controlled access mechanism for shared resources.

The arbiter takes 4 input requests and outputs a single grant in the form of ONE HOT

Round Robin scheduling $REQ[0] < REQ[1] < REQ[2] < REQ[3] < REQ[0] \dots$

Comprehensive insights into the topic are available within the contents of [1].

ROUND ROBIN ALGORITHM:

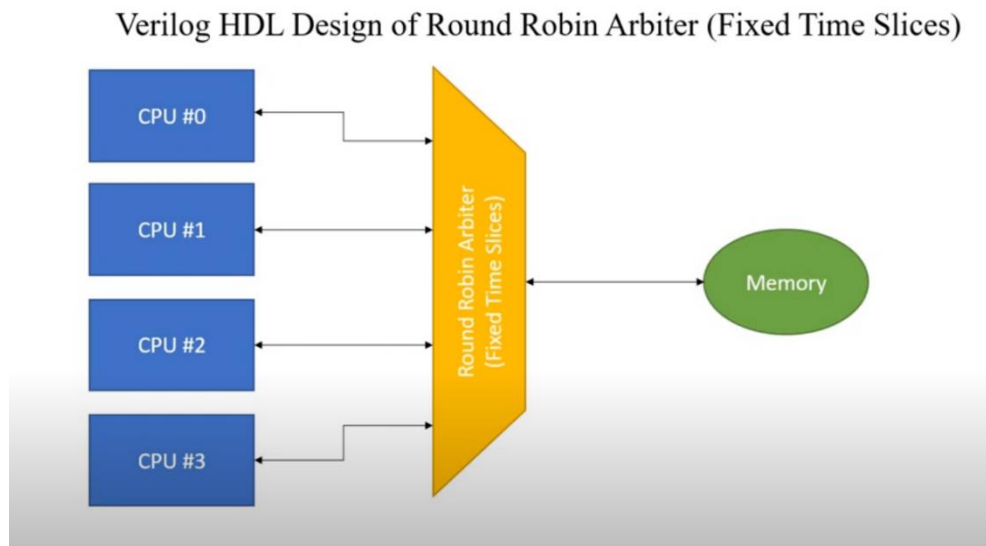
The Round Robin Algorithm, a flexible scheduling approach, has found applications in diverse computing contexts, including CPU scheduling, shared bus arbitration, queue management, and workload balancing. This algorithm is particularly known for its ease of implementation and its effectiveness in preventing process starvation.

Round Robin in CPU Scheduling:

Functioning as a CPU scheduling algorithm, Round Robin allocates a fixed time slot to each process in the queue. This cyclic time-sharing ensures equitable and timely service for each requestor. A notable advantage of Round Robin is its ability to address process starvation, a common concern with fixed priority algorithms. In contrast to Fixed Priority Algorithms, where access to the lowest priority process has an unlimited time interval, Round Robin introduces a cyclic order. Every process request is granted within a fixed time slot as facilitated by a designated pointer, promoting fair resource allocation.

Bus Arbitration, Queueing, and Workload Balancing: Beyond CPU scheduling, Round Robin is extensively utilized in shared bus arbitration, queue management, and workload balancing. In shared bus arbitration, the cyclic nature of the algorithm ensures that each requestor is served and

then moved to the end of the queue, preventing any single requestor from monopolizing the bus. This characteristic makes Round Robin an attractive option in scenarios where fair and equal access to shared resources is crucial.



Challenges and Considerations:

While Round Robin effectively addresses process starvation and promotes fairness in resource allocation, it is important to acknowledge certain challenges. The maximum wait period before a request is re-served is inherently tied to the number of requestors in the queue. This introduces variability in response times based on system load, requiring careful consideration to balance fairness and efficiency.

In summary, the Round Robin Algorithm emerges as a robust and widely applicable scheduling technique. Its adoption in CPU scheduling, bus arbitration, queue management, and workload balancing highlight its versatility. However, the algorithm's performance is intricately linked to the number of requestors in the queue, emphasizing the need for a nuanced approach to system design and resource allocation.

DESIGN:***Design Choice:***

There are two ways to design the round robin arbiter and that was using either nested case statements or using pointer. Here we have designed the arbiter using **case statement**. This method was chosen due to 2 reasons:

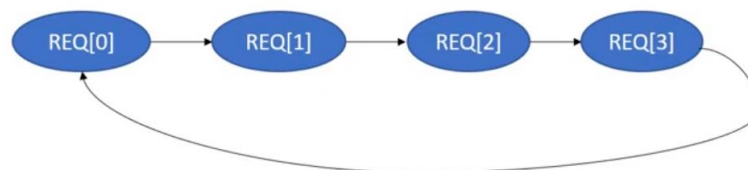
1. The case statement method is practical for a limited small no. of inputs whereas the pointer method is for large no. of inputs. In our case we have 4 input requestors which is not a lot hence using case statement method was reasonable.
2. The case statement is actually easier to understand, code and implement hence making it a better choice for our design.

Structure of Arbiter Module:

Our design of round robin arbiter takes 4 REQUEST's which are 4 bits wide each in addition to that clock and reset signals are also used as inputs in our arbiter. Furthermore, the arbiter has a single GRANT output which will also be 4 bits wide.

***Flow of Data:***

On every positive edge of clock, a request will be granted the resource in a fixed time slot. It works on the principle of first come first serve basis and which ever request arrives first whether it be REQ [0], [1] or [3] the first one will get entertained first. However, the check for incoming request will always start from the first request i.e. REQ [0]



VERILOG HDL CODE:

```

1 module roundrobin_fixedtimeslice(
2     input clk,
3     input rst_n,
4     input [3:0] REQ,
5     output reg [3:0] GNT
6 );
7
8     reg [2:0] present_state;
9     reg [2:0] next_state;
10    parameter [2:0] S_ideal = 3'b000;
11    parameter [2:0] S_0 = 3'b001;
12    parameter [2:0] S_1 = 3'b010;
13    parameter [2:0] S_2 = 3'b011;
14    parameter [2:0] S_3 = 3'b100;
15
16
17    always @ (posedge clk or negedge rst_n) // State Register , Sequential always block
18    begin
19        if(!rst_n)
20            present_state <= S_ideal;
21        else
22            present_state <= next_state;
23        end
24
25    always @(*) // Next State , Combinational always block
26    begin
27        case(present_state)
28
29            S_ideal : begin
30                if(REQ[0])
31                begin
32                    next_state = S_0;
33                end
34
35                else if(REQ[1])
36                begin
37                    next_state = S_1;
38                end
39
40                else if(REQ[2])
41                begin
42                    next_state = S_2;
43                end
44
45                else if(REQ[3])
46                begin
47                    next_state = S_3;
48                end
49
50                else
51                begin
52                    next_state = S_ideal;
53                end
54
55                end // S_ideal
56
57            S_0 : begin
58                if(REQ[1])
59                begin
60                    next_state = S_1;
61                end
62
63                else if(REQ[2])
64                begin
65                    next_state = S_2;
66                end
67
68                else if(REQ[3])
69                begin
70                    next_state = S_3;
71                end
72
73                else if(REQ[0])
74                begin
75                    next_state = S_0;
76                end
77
78                else
79                begin
80                    next_state = S_ideal;
81                end
82
83                end // S_0

```

```

84
85
86 S_1 : begin
87     if(REQ[2])
88     begin
89         next_state = S_2;
90     end
91
92     else if(REQ[3])
93     begin
94         next_state = S_3;
95     end
96
97     else if(REQ[0])
98     begin
99         next_state = S_0;
100     end
101
102     else if(REQ[1])
103     begin
104         next_state = S_1;
105     end
106
107     else
108     begin
109         next_state = S_ideal;
110     end
111
112 end //S_1
113
114 S_2 : begin
115     if(REQ[3])
116     begin
117         next_state = S_3;
118     end
119
120     else if(REQ[0])
121     begin
122         next_state = S_0;
123     end
124
125     else if(REQ[1])
126     begin
127         next_state = S_1;
128     end
129
130     else if(REQ[2])
131     begin
132         next_state = S_2;
133     end
134
135     else
136     begin
137         next_state = S_ideal;
138     end
139
140 end // S_2
141
142 S_3 : begin
143     if(REQ[0])
144     begin
145         next_state = S_0;
146     end
147
148     else if(REQ[1])
149     begin
150         next_state = S_1;
151     end
152
153     else if(REQ[2])
154     begin
155         next_state = S_2;
156     end
157
158     else if(REQ[3])
159     begin
160         next_state = S_3;
161     end
162
163     else
164     begin
165         next_state = S_ideal;
166     end
167
168 end // S_3
169
170 default : begin
171     if(REQ[0])
172     begin
173         next_state = S_0;
174     end
175
176     else if(REQ[1])

```

```

176         begin
177             next_state = S_1;
178         end
179
180         else if(REQ[2])
181         begin
182             next_state = S_2;
183         end
184
185         else if(REQ[3])
186         begin
187             next_state = S_3;
188         end
189
190         else
191         begin
192             next_state = S_ideal;
193         end
194
195         end // default
196     endcase // case(state)
197
198 end
199
200
201
202
203 always @(*) // Output , Combinational always block
204 begin
205     case(present_state)
206     S_0 : begin GNT = 4'b0001; end
207     S_1 : begin GNT = 4'b0010; end
208     S_2 : begin GNT = 4'b0100; end
209     S_3 : begin GNT = 4'b1000; end
210     default : begin GNT = 4'b0000; end
211
212     endcase
213
214 end
215
216 endmodule // Round Robin Arbiter with Fixed Time Slice
217

```

Test bench:

```

1 module roundrobin_tb;
2     reg clk;
3     reg rst_n;
4     reg [3:0] REQ;
5     wire [3:0] GNT;
6
7     //Instantiate Design Under Test
8
9     roundrobin_fixedtimeslice DUT(.clk(clk), .rst_n(rst_n), .REQ(REQ), .GNT(GNT));
10
11     //Generate a 10 ns Time Period Clock
12     always #5 clk = ~clk;
13
14     //Drive the DUT or Generate stimuli for the DUT
15
16     initial begin
17         clk = 0;
18         rst_n = 1;
19         REQ = 4'b0;
20         // Assert the Asynchronous Reset after 1 clock period
21         #10 rst_n = 0;
22         //Deassert the Reset
23         #5 rst_n = 1;
24
25         @(negedge clk) REQ = 4'b0010; //REQ[1]
26         @(negedge clk) REQ = 4'b1110; //REQ[2]
27         @(negedge clk) REQ = 4'b0010; //REQ[1]
28         @(negedge clk) REQ = 4'b1010; //REQ[3]
29         @(negedge clk) REQ = 4'b0100; //REQ[2]
30         @(negedge clk) REQ = 4'b1111; //REQ[3]
31         @(negedge clk) REQ = 4'b0110; //REQ[1]
32         @(negedge clk) REQ = 4'b1000; //REQ[3]
33         @(negedge clk) REQ = 4'b0010; //REQ[1]
34         @(negedge clk) REQ = 4'b1010; //REQ[3]
35         @(negedge clk) REQ = 4'b0100; //REQ[2]
36         #5 rst_n = 0;
37
38     end
39 endmodule

```


EXPLANATION OF CODE:

In order to design the arbiter, we have implemented the logic using a Finite State Machine which comprises of 4 states S_ideal, S_0, S_1, S_2, S_3. In order to perform transition between these states we have assigned two variables/registers named present state and next state.

Whenever there is a positive edge of clock or a negative edge of reset, the value of next state is assigned to the present state register hence the state switches in the state machine, however if reset signal is pulled LOW the present state will be assigned as S_ideal state

State Transitions:

The state machine is constructed via case statement which operates when there is a change in the variable present state.

Assuming the present state is at S_ideal state at reset=0, which later switches to reset =1 as input is received. Now during the ideal state if a requester is requesting from source for e.g. if REQ [2] is at input, the if else conditions in the S_ideal block will run in the default sequence i.e. REQ [0] ... REQ [3]. And as the corresponding conditional statement of REQ [2] is executed the value of S_2 will be assigned to the next state register. Furthermore, on positive edge of clock the value of next state will be assigned to the present state register hence resulting in the change of state to S_2. In the S_2 state the REQ [3] will have the maximum priority and the sequence will be; REQ [3], REQ [0], REQ [1], REQ [2].

Now the state machine will wait for next requester and depending on the requester it will assign the corresponding state to the present state variable, however if no requests are pulled the present state will switch to the S_ideal state with the default sequence.

The requester gets entertained whenever its corresponding state is at present state i.e. the value of grant will be assigned the requestor.

Setting up Requests in Test Bench:

Here we have constructed a test bench which generates an input clock of 10ps. The initial value of reset is assigned 1 and after 10ps delay the value of reset is assigned 0 which starts the S_ideal state at this point:

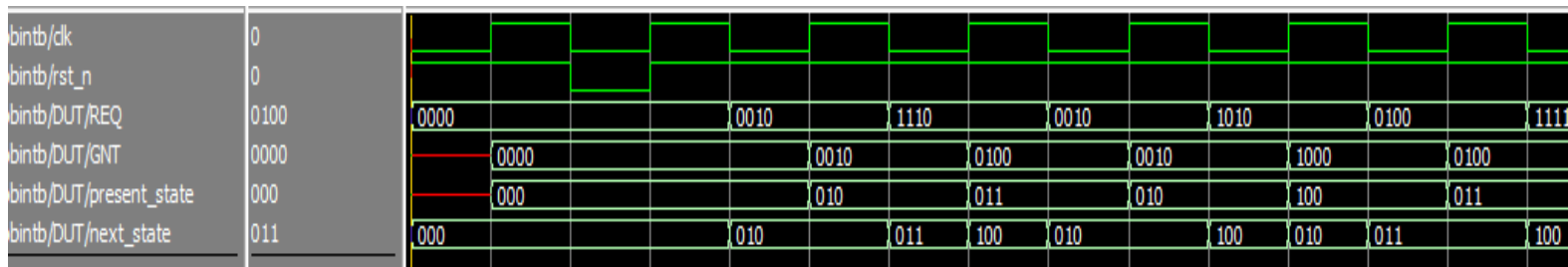
Present state = S_ideal

The first requester that is requesting in our test bench is the REQ [1] with the value 4'b0010. when the request arrives the corresponding if else statement of REQ [1] is executed and the present state variable is assigned the value S_1 and as the state changes to S_1 the value of REQ [1] is assigned to the GRANT in other words the request is granted.

After that these other requests are also granted in the same manner in a fixed time slot for each of them (fixed time slice).

The output waveform of the test bench is shown below:

OUTPUT WAVEFORM:



CONCLUSION:

After analyzing the output waveform for our design of Round Robin Arbiter we found out that when we provide any of the 4 REQUESTS in any combination possible, the requests of the corresponding inputs are granted in a fixed time interval/slice i.e. the resource is granted to all the requests equally on their fixed time and no priority is given to any of the inputs, therefore there is no starvation of requests in our design which we can confirm from our output waveform.

Hence, we can conclude that our design meets the expectation of the Round Robin Arbiter and fulfill the requirements of the problem statement.

REFERENCES:

- [1] A. Mishra, *Design of a Round Robin Bus Arbiter using System Verilog*, 2020.